Misremembered Media — Implementation Plan

For: Google Antigravity
Source: `index.html` + `app.js` (current build)

---

0. Reference behavior (from provided image sequence)

Across repeated exposure to the same source, the pattern is consistent:

- The **tileable base texture** (wallpaper) is essentially never corrupted — same color, same repeat, same striping the whole way through.
- **Discrete salient features** (the "Backrooms Original Soundtrack" text, the speaker box, the pipe hinge junction) are what drift: mirrored, duplicated as an overlapping echo, slowly rotated/tilted off their original angle, or duplicated as a second symmetric copy.
- The end state isn't more corruption — it's **erasure**. The final frame in the sequence has no text, no box, no hinge, nothing but plain wallpaper. The

> Complex didn't destroy the image, it just stopped remembering the specific things and kept the generic thing.

This means the correct model is **per-entity drift with an eventual fade to absence**, layered over an untouched base — not a pixel/region operation applied to the whole frame. This replaces the block-slice approach completely.

---

1. REMOVE: block-slice geometry shear

Delete `applyGeometryShear()` in full, and remove its call site from the render pipeline. Also remove `applyObjectStretch()`'s current implementation (the random-rect + `drawImage` scale-up) — same failure mode, different function: it treats "stretch" as one flat scale operation on an anonymous rectangle instead of a tracked object drifting over time.

Why these fail structurally, for the record: both operate on **anonymous, memoryless regions** — a new random rect (or band) is chosen every call, scaled/shifted as one flat block, with hard edges (solid black fill or clipped rect) instead of blending into the surrounding frame. There's no concept of "this is the same object as last time, and it's a little more wrong now." That's

the whole thing your reference images are showing, so it can't be patched — it needs the entity model below.

---

2. NEW CORE SYSTEM: Entity Memory Model

This is the foundation everything else (including your original ideas) should hook into.

2.1 Feature detection (one-time per source load)

Extend the existing `scanForTextCandidates(w, h)` — it already finds text-like high-contrast rectangular regions. Add a sibling `scanForObjectCandidates(w, h)` using the same salience approach (edge-density / contrast blocks) but tuned for larger, lower-frequency regions (furniture, hardware, fixtures) rather than tight text blocks. Run both once when a file is loaded, not per-frame.

2.2 Entity representation

```
// One entry per detected salient feature,
persists for the life of the loaded source
let memoryEntities = [];
// entity shape:
```

```
{
  id,
  type: 'text' | 'object' | 'hardware',
  baseRect: {x, y, w, h},        //
original detected position, never mutated
  decayLevel: 0,                  //
increments once per loop/repetition
  transform: {
    mirrorX: false,
    mirrorY: false,
    rotation: 0,                  //
radians, drifts with decayLevel
    duplicated: false,
    dupOffset: {x: 0, y: 0},
    opacity: 1.0                  // ramps
to 0 as decayLevel climbs
  }
}
```

2.3 Repetition tracking (the decay trigger)

Hook into `sourceVideo`'s loop event (or your existing seek-to-0 / replay-start logic near `startReplay` / `handleEndFade`). On each detected repetition:

1. `decayLevel++` globally, or per-entity if you want some features to degrade faster than others (recommended — text should mirror/duplicate before hardware does, hardware should duplicate before it fades, matching the reference

sequence's apparent order: text corrupts first, geometry drifts second, erasure last).

2. Pick 1–2 entities via `frameRng` (reuse your existing seeded RNG so this is reproducible from a seed) and apply exactly one mutation step to each, chosen from the pool below, weighted by current `decayLevel`:
    - **Low decay** → `mirrorX` or `mirrorY` flip (text-first behavior, matches images 2/5/9)
    - **Mid decay** → `duplicated = true` with a small `dupOffset` and reduced opacity on the duplicate (matches images 2/9's overlapping echo, image 6's hinge duplication)
    - **Mid-high decay** → `rotation` increments by a small random amount each hit (matches image 3's tilted box — this should accumulate, not jump)
    - **High decay** → `opacity` ramps down; at `opacity <= 0` the entity stops rendering entirely (matches image 7's total erasure)

Mutations should **compound**, not reset — an entity that's already mirrored and then hits a duplicate-mutation should render as a mirrored duplicate, which is exactly what image 9 shows (two overlapping copies, both partially mirrored).

2.4 Render function

```
function renderMemoryEntities(ctx, now) {
  for (const e of memoryEntities) {
    if (e.transform.opacity <= 0) continue;
// fully forgotten
    ctx.save();
    ctx.globalAlpha = e.transform.opacity;
    const cx = e.baseRect.x + e.baseRect.w
/ 2;
    const cy = e.baseRect.y + e.baseRect.h
/ 2;
    ctx.translate(cx, cy);
    ctx.rotate(e.transform.rotation);
    ctx.scale(e.transform.mirrorX ? -1 : 1,
e.transform.mirrorY ? -1 : 1);
    ctx.translate(-cx, -cy);
    // draw the region from the
clean/current source frame into place
    ctx.drawImage(sourceCanvasOrFrame,
e.baseRect.x, e.baseRect.y, e.baseRect.w,
e.baseRect.h,
                  e.baseRect.x,
e.baseRect.y, e.baseRect.w, e.baseRect.h);
    ctx.restore();

    if (e.transform.duplicated) {
      // second pass, offset, lower
opacity, same transform — the "echo" copy
      ctx.save();
      ctx.globalAlpha = e.transform.opacity
* 0.55;
      ctx.translate(e.dupOffset.x,
e.dupOffset.y);
      // repeat the same draw with the same
```

```
transform stack
      ctx.restore();
    }
  }
}
```

Key point: this draws the entity **on top of** the untouched base frame — the base texture underneath is never touched by this system, which is what preserves the "wallpaper never corrupts" behavior from your reference images. Feather the edges of each entity's rect with a small (4–8px) gradient alpha mask so mirrored/rotated copies don't look pasted-in with hard seams.

---

3. Your original ideas, folded in

3.1 No-signal → Complex-generated feed → back

- Extend `activeVisualEvent.type` with `'complex_generated'`.
- Schedule via `generateDistortionSchedule` alongside existing `no_signal` events.
- New render function `renderComplexGeneratedFrame(w, h, now)`: reuse `renderNoSignalScreen`'s canvas setup, but instead of the "NO SIGNAL" text, generate

genuinely new geometry — procedural corridor/room outline via perspective lines + tiled pattern fill using `frameRng`, so it visibly isn't your source, it's the Complex's own output.

- Audio: new `playComplexGeneratedAmbience()`, same oscillator/gain infrastructure as `playNoSignalMusic()` but distinct harmonic content so it reads as a different "voice," not a reused cue.
- Transition: crossfade in/out (~150–300ms alpha ramp) rather than a hard cut, using your existing `vhsTrackingJitterUntil`-style pre-roll (rename it — no VHS framing anymore, just "reality glitch roll").

3.2 Music substitution (generated / recurring motif)

- New `createComplexGeneratedMusicSynth()` alongside `createWhisperSynthesizer()` — layered detuned oscillators + slow LFO panning, fully synthesized (not sampled/reproduced tracks — keeps this original and copyright-clean).
- Maintain 2–3 fixed "motif" definitions seeded from `currentSeed`, so the same recurring one keeps resurfacing across a session — same reproducibility approach as the rest of your seed system.

- Trigger via a new `music_substitution` event type in the schedule; duck real audio via `mainGainNode`, ramp up the motif's gain node, crossfade back on end.

3.3 Forgetting objects → inserting known objects

- This is now naturally a special case of the entity model: an **inserted entity** starts at `decayLevel = 0`, `opacity` ramping *up* from 0 instead of down, type `'inserted'`.
- Render as simple procedural silhouettes (rect/path primitives — chair back, speaker cone, cabinet edge), not raster images, to stay visually consistent with "the Complex approximating from limited reference."
- Preferentially place these on a `scanForObjectCandidates` region that's already faded out (`opacity <= 0`) — i.e., it replaces something it just forgot, which is a nice closed loop with 2.3's erasure step.

---

4. Build order for Antigravity

1. Remove `applyGeometryShear` / old `applyObjectStretch` and their call sites. (Isolated, low-risk, do first.)

2. Build the entity detection + data model (2.1–2.2), no rendering yet — verify detected regions visually (draw debug outlines) before wiring mutations.
3. Build `renderMemoryEntities` + the four mutation types (2.4), triggered manually via a temporary debug button per mutation type (mirror / duplicate / rotate / fade) so each can be previewed on demand instead of waiting on the loop-count trigger.
4. Wire the repetition/decay trigger (2.3) to actual playback looping.
5. Object insertion (3.3) — reuses the entity system, minimal new code.
6. Music substitution (3.2) — self-contained audio graph work.
7. No-signal generated feed (3.1) — most complex, composes visual + audio + scheduler, do last.

5. Testing notes

Use `resetSeed()` with a fixed value to get a reproducible decay sequence for QA instead of waiting on random playback. Worth adding a debug readout of each entity's current `decayLevel` / `transform` state next to your existing

log terminal while building this, then stripping it before ship.